

Module 10 Notes: Deleting from Singly Linked Lists

1. Current Progress Analysis

You are ready for Module 10 because you already know the main ideas needed for deletion.

From Module 1: Arrays vs Linked Lists

You learned why linked lists exist.

Arrays are fast when we already know the index, but arrays have a fixed size after creation. Linked lists are more flexible because they can grow and shrink one node at a time.

Structure	Strength	Weakness
Array	Fast direct access by index	Fixed size and shifting may be needed during insertion or deletion
Linked list	Flexible size and easier pointer-based insertion or deletion	No direct jumping to an index

This matters for Module 10 because deletion in an array may require shifting values, but deletion in a linked list mainly requires changing pointers.

From Module 2: Node Anatomy and Pointer Movement

You learned that a singly linked-list node has two parts:

[data | next]

Part	Meaning
data	The value stored in the node
next	The address of the next node

C node definition:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

You also learned the most important pointer movement:

```
p = p->next;
```

This moves pointer `p` to the next node.

Module 10 depends heavily on:

```
p = p->next;  
q = p;  
p != NULL;  
p->data;  
p->next;
```

From Module 3: Traversal and Display

You learned traversal.

Traversal means visiting nodes one by one from the first node until `NULL`.

Example:

```
first -> 10 -> 20 -> 30 -> NULL
```

Basic traversal pattern:

```
while (p != NULL) {  
    // use p->data  
    p = p->next;  
}
```

This matters because deletion by index requires moving through the list until we reach the node we want to delete.

From Module 5: Count Function

You learned how to count nodes in a linked list.

The `count()` function is important in Module 10 because we use it to check whether a deletion index is valid.

```

int count(struct Node *p) {
    int c = 0;

    while (p != NULL) {
        c++;
        p = p->next;
    }

    return c;
}

```

From Module 6 and Module 9: Using `p` and `q`

You learned that some linked-list operations need two pointers.

Pointer	Common meaning
<code>p</code>	Current pointer
<code>q</code>	Previous or trailing pointer

This is very important in deletion.

When deleting a middle or last node:

- `p` points to the node we want to delete.
- `q` points to the node before `p`.

From Module 7: Link Changing

You learned insertion by changing links.

Insertion connects a new node into the list.

Deletion does the opposite:

Insertion: create a node and connect it.
 Deletion: disconnect a node and free it.

So Module 10 continues the same pointer idea, but now we remove nodes instead of adding them.

2. Big Idea of Module 10

Module 10 teaches how to remove a node from a singly linked list.

Example list:

```
10 -> 20 -> 30 -> 40 -> 50 -> NULL
```

If we delete the node containing `40`, the result should be:

```
10 -> 20 -> 30 -> 50 -> NULL
```

The important idea is:

```
Deleting a node does not mean shifting values.  
Deleting a node means changing links, then freeing memory.
```

In an array, deleting an element may require moving many values.

In a linked list, we do not shift values. We make one node skip over another node.

Example:

Before deleting `40`:

```
30 -> 40 -> 50
```

After changing the link:

```
30 -----> 50
```

Then we release the memory used by node `40`.

3. What Does It Mean to Delete a Node?

Deleting a node has two parts.

Part 1: Remove the node from the list chain

This means changing links so the previous node skips the deleted node.

Example:

```
Before:  
10 -> 20 -> 30 -> NULL
```

If we delete `20`, we make `10` point directly to `30`.

```
After link change:  
10 -----> 30 -> NULL
```

Part 2: Free the removed node from memory

The removed node was created in heap memory using `malloc`.

So after removing it from the chain, we must release its memory using:

```
free(p);
```

Complete idea:

```
Change the links first.  
Then free the removed node.
```

4. Deletion Index Convention

Module 10 uses 1-based indexing for deletion.

This is different from the insertion convention from Module 7.

For deletion:

```
index 1 = delete the first node  
index 2 = delete the second node  
index 3 = delete the third node
```

index 4 = delete the fourth node
...

Example list:

10 -> 20 -> 30 -> 40 -> 50 -> NULL

Delete index	Node deleted	Final list
1	10	20 -> 30 -> 40 -> 50 -> NULL
2	20	10 -> 30 -> 40 -> 50 -> NULL
3	30	10 -> 20 -> 40 -> 50 -> NULL
4	40	10 -> 20 -> 30 -> 50 -> NULL
5	50	10 -> 20 -> 30 -> 40 -> NULL

If a list has `n` nodes, valid deletion indexes are:

1 to n

So if the list has 5 nodes, valid indexes are:

1, 2, 3, 4, 5

Invalid indexes include:

0, -1, 6, 100

Important rule:

For deletion, index 0 is invalid.

This is different from insertion, where index `0` meant inserting before the first node.

5. Why Index Validation Is Needed

Before deleting, we must check whether the index is valid.

The validation rule is:

```
if (index < 1 || index > count(p)) {  
    return -1;  
}
```

Meaning:

Condition	Meaning
<code>index < 1</code>	Index is too small. Deletion starts at index 1.
<code>index > count(p)</code>	Index is larger than the number of nodes.

If the index is invalid, the function returns:

-1

and the list is not changed.

Example: Invalid index 0

List:

10 -> 20 -> 30 -> NULL

Call:

```
Delete(first, 0);
```

Since index `0` is invalid, the function returns:

-1

The list remains:

10 -> 20 -> 30 -> NULL

Example: Invalid index too large

List:

```
10 -> 20 -> 30 -> NULL
```

The list has 3 nodes.

Call:

```
Delete(first, 8);
```

Index `8` is invalid because there is no 8th node.

The function returns:

```
-1
```

The list remains unchanged.

6. The Two Main Deletion Cases

There are two main cases in singly linked-list deletion:

1. Delete the first node.
2. Delete any other node.

Why are there two cases?

Because deleting the first node changes `first` itself.

Deleting a middle or last node does not move `first`; it changes the `next` pointer of the previous node.

7. Case 1: Deleting the First Node

Suppose the list is:


```
first
|
v
10 -> 20 -> 30 -> NULL
```

We want to delete index `1`, which means delete node `10`.

Step 1: Save the first node in `q`

```
q = first;
```

Now both `first` and `q` point to node `10`.

```
q
|
v
10 -> 20 -> 30 -> NULL
^
|
first
```

Why do we need `q`?

Because after we move `first`, we still need a pointer to the old first node so we can free it.

Step 2: Save the deleted value

```
x = q->data;
```

Now:

```
x = 10
```

We save the value before freeing the node because after freeing, we should not access the node.

Step 3: Move `first` to the second node

```
first = first->next;
```

Now `first` points to `20`.

```

q
|
v
10 -> 20 -> 30 -> NULL
      ^
      |
    first

```

The list now starts from **20**.

Step 4: Free the old first node

```
free(q);
```

Node **10** is released from heap memory.

Final list:

```

first
|
v
20 -> 30 -> NULL

```

Returned value:

```
10
```

Head Deletion Code Pattern

```

q = first;
x = q->data;
first = first->next;
free(q);
return x;

```

Important rule:

```
Move first before freeing the old first node.
```

Wrong order:

```
free(first);  
first = first->next; // unsafe
```

This is unsafe because after `free(first)`, using `first->next` is invalid.

8. Case 2: Deleting a Middle Node

Suppose the list is:

```
10 -> 20 -> 30 -> 40 -> 50 -> NULL
```

We want to delete index `4`.

Index `4` means delete node `40`.

To delete a middle node, we need two pointers:

Pointer	Meaning
<code>p</code>	Points to the node we want to delete
<code>q</code>	Points to the node before <code>p</code>

At the correct position:

```
10 -> 20 -> 30 -> 40 -> 50 -> NULL  
          q      p
```

The goal is to make `q` skip over `p`.

Important line

```
q->next = p->next;
```

Here:

- `q` points to `30`.
- `p` points to `40`.
- `p->next` points to `50`.

So this line means:

Make 30 point to 50.

Before:

```
30 -> 40 -> 50
q      p
```

After:

```
30 -----> 50
q
```

Now node 40 is removed from the list chain.

Then we save its data and free it:

```
x = p->data;
free(p);
return x;
```

Final list:

```
10 -> 20 -> 30 -> 50 -> NULL
```

Returned value:

40

9. Case 3: Deleting the Last Node

Deleting the last node uses the same logic as deleting a middle node.

Example list:

```
10 -> 20 -> 30 -> NULL
```

Call:

```
Delete(first, 3);
```

Index `3` means delete node `30`.

At the correct position:

```
10 -> 20 -> 30 -> NULL
      q      p
```

Now:

```
q->next = p->next;
```

Since `p->next` is `NULL`, this becomes:

```
q->next = NULL;
```

So node `20` becomes the new last node.

Final list:

```
10 -> 20 -> NULL
```

Returned value:

```
30
```

Important idea:

```
Deleting the last node means the previous node's next becomes NULL.
```

10. How to Move `p` and `q` to the Correct Position

For deletion after index `1`, we use a loop.

```
for (int i = 0; i < index - 1; i++) {
    q = p;
    p = p->next;
}
```

At the start:

```
p = first;
q = NULL;
```

Each loop movement does two things:

```
q = p;
p = p->next;
```

Meaning:

1. **q** moves to the current node.
2. **p** moves to the next node.

So **q** stays one node behind **p**.

Example: Delete index 4

List:

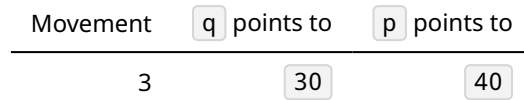
```
10 -> 20 -> 30 -> 40 -> 50 -> NULL
```

Start:

```
q = NULL
p = 10
```

Loop movements:

Movement	q points to	p points to
Start	NULL	10
1	10	20
2	20	30



After 3 movements:

```
10 -> 20 -> 30 -> 40 -> 50 -> NULL
      q      p
```

Now **p** points to the node to delete.

11. Full Delete Function in C

```
int Delete(struct Node *p, int index) {
    struct Node *q = NULL;
    int x = -1;

    if (index < 1 || index > count(p)) {
        return -1;
    }

    if (index == 1) {
        q = first;
        x = q->data;
        first = first->next;
        free(q);
        return x;
    }

    for (int i = 0; i < index - 1; i++) {
        q = p;
        p = p->next;
    }

    q->next = p->next;
    x = p->data;
    free(p);

    return x;
}
```

12. Explanation of the Delete Function

Function Header

```
int Delete(struct Node *p, int index)
```

Meaning:

Part	Meaning
<code>int</code>	The function returns the deleted value.
<code>struct Node *p</code>	Pointer used to traverse the list.
<code>int index</code>	The position of the node to delete.

The function returns:

- the deleted value if deletion succeeds
- `-1` if the index is invalid

Local Variables

```
struct Node *q = NULL;  
int x = -1;
```

Variable	Meaning
<code>q</code>	Previous pointer or temporary pointer
<code>x</code>	Stores the deleted value

Validation

```
if (index < 1 || index > count(p)) {  
    return -1;  
}
```

This prevents invalid deletion.

If the index is invalid, the function stops immediately and returns `-1`.

First Node Case

```
if (index == 1) {
    q = first;
    x = q->data;
    first = first->next;
    free(q);
    return x;
}
```

This deletes the first node.

Why is it separate?

Because `first` must be updated.

Middle or Last Node Case

```
for (int i = 0; i < index - 1; i++) {
    q = p;
    p = p->next;
}

q->next = p->next;
x = p->data;
free(p);

return x;
```

This moves `p` to the node to delete and `q` to the node before it.

Then it removes `p` from the chain and frees it.

13. Full Working C Program for Module 10

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};
```

```

struct Node *first = NULL;

void create(int A[], int n) {
    if (n <= 0) {
        first = NULL;
        return;
    }

    struct Node *t;
    struct Node *last;

    first = (struct Node *)malloc(sizeof(struct Node));
    first->data = A[0];
    first->next = NULL;
    last = first;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;

        last->next = t;
        last = t;
    }
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}

int count(struct Node *p) {
    int c = 0;

    while (p != NULL) {
        c++;
        p = p->next;
    }

    return c;
}

int Delete(struct Node *p, int index) {
    struct Node *q = NULL;
    int x = -1;

```

```

    if (index < 1 || index > count(p)) {
        return -1;
    }

    if (index == 1) {
        q = first;
        x = q->data;
        first = first->next;
        free(q);
        return x;
    }

    for (int i = 0; i < index - 1; i++) {
        q = p;
        p = p->next;
    }

    q->next = p->next;
    x = p->data;
    free(p);

    return x;
}

int main() {
    int A[] = {10, 20, 30, 40, 50};

    create(A, 5);

    printf("Original list: ");
    display(first);

    int deleted = Delete(first, 4);
    printf("\nDeleted value: %d", deleted);

    printf("\nAfter deleting index 4: ");
    display(first);

    deleted = Delete(first, 1);
    printf("\nDeleted value: %d", deleted);

    printf("\nAfter deleting index 1: ");
    display(first);

    deleted = Delete(first, 8);
    printf("\nDeleted value for invalid index: %d", deleted);

    printf("\nAfter invalid deletion: ");

```

```
    display(first);  
  
    return 0;  
}
```

Expected output:

```
Original list: 10 20 30 40 50  
Deleted value: 40  
After deleting index 4: 10 20 30 50  
Deleted value: 10  
After deleting index 1: 20 30 50  
Deleted value for invalid index: -1  
After invalid deletion: 20 30 50
```

14. Dry Run 1: Delete First Node

Original list:

```
10 -> 20 -> 30 -> NULL
```

Call:

```
Delete(first, 1);
```

Step 1: Validate index

The list has 3 nodes.

Valid indexes are:

```
1, 2, 3
```

Index is valid.

Step 2: Make point to first

```
q = first;
```

Now:

```
q
|
v
10 -> 20 -> 30 -> NULL
^
|
first
```

Step 3: Save the deleted value

```
x = q->data;
```

So:

```
x = 10
```

Step 4: Move `first`

```
first = first->next;
```

Now:

```
10 -> 20 -> 30 -> NULL
      ^
      |
      first
```

Step 5: Free the old first node

```
free(q);
```

Node `10` is removed from memory.

Final list:

```
20 -> 30 -> NULL
```

Returned value:

10

15. Dry Run 2: Delete Index 4

Original list:

10 -> 20 -> 30 -> 40 -> 50 -> NULL

Call:

```
Delete(first, 4);
```

Step 1: Validate index

The list has 5 nodes.

Valid indexes are:

1, 2, 3, 4, 5

Index 4 is valid.

Step 2: Move p and q

Start:

```
p = 10  
q = NULL
```

Loop movement table:

Movement	q points to	p points to
Start	NULL	10
1	10	20

Movement	q points to	p points to
2	20	30
3	30	40

Now:

```
10 -> 20 -> 30 -> 40 -> 50 -> NULL
      q      p
```

Step 3: Make q skip p

```
q->next = p->next;
```

This makes node 30 point to node 50.

```
10 -> 20 -> 30 -----> 50 -> NULL
```

Step 4: Save the deleted value

```
x = p->data;
```

So:

```
x = 40
```

Step 5: Free the deleted node

```
free(p);
```

Node 40 is released from memory.

Final list:

```
10 -> 20 -> 30 -> 50 -> NULL
```

Returned value:

40

16. Dry Run 3: Delete Last Node

Original list:

10 -> 20 -> 30 -> NULL

Call:

```
Delete(first, 3);
```

Step 1: Validate index

The list has 3 nodes.

Valid indexes are:

1, 2, 3

Index 3 is valid.

Step 2: Move p and q

After moving:

10 -> 20 -> 30 -> NULL
 q p

Step 3: Make q skip p

```
q->next = p->next;
```

Since p->next is NULL, this makes:

```
q->next = NULL;
```


So node `20` becomes the last node.

Final list:

```
10 -> 20 -> NULL
```

Returned value:

```
30
```

17. Dry Run 4: Invalid Index

Original list:

```
10 -> 20 -> 30 -> NULL
```

Call:

```
Delete(first, 0);
```

Validation:

```
if (index < 1 || index > count(p)) {  
    return -1;  
}
```

Since:

```
0 < 1
```

index `0` is invalid.

The function returns:

```
-1
```

The list remains unchanged:

```
10 -> 20 -> 30 -> NULL
```

Important idea:

```
Invalid deletion should not damage the list.
```

18. Dry Run 5: Delete from a One-Node List

Original list:

```
10 -> NULL
```

Call:

```
Delete(first, 1);
```

Since index **1** means delete the first node, we use the head deletion case.

Step 1:

```
q = first;
```

Now **q** points to node **10**.

Step 2:

```
x = q->data;
```

So:

```
x = 10
```

Step 3:

```
first = first->next;
```

Since `first->next` is `NULL`, now:

```
first = NULL
```

Step 4:

```
free(q);
```

Node `10` is freed.

Final list:

```
NULL
```

Returned value:

```
10
```

Important:

```
Deleting the only node makes the list empty.
```

19. Empty List Case

An empty list means:

```
first = NULL
```

If we call:

```
Delete(first, 1);
```

Then `count(first)` is `0`.

Validation checks:

```
index > count(p)
```

Since:

```
1 > 0
```

The index is invalid.

The function returns:

```
-1
```

The list remains empty.

20. The Most Important Pointer Line in Deletion

For non-head deletion, the most important line is:

```
q->next = p->next;
```

Meaning:

```
The previous node skips the target node.
```

Example:

```
Before:  
30 -> 40 -> 50  
q     p
```

After:

```
30 -----> 50  
q
```

This line removes `p` from the chain.

But this line does not free memory.

So complete deletion needs:

```
q->next = p->next; // remove from list chain
free(p);           // release heap memory
```

Key idea:

Skipping a node and freeing a node are not the same thing.

21. Why We Save the Deleted Value

The delete function returns the value that was deleted.

Example:

```
int deleted = Delete(first, 4);
```

If the deleted node stores `40`, then:

```
deleted = 40
```

To do this safely, we save the data before freeing the node:

```
x = p->data;
free(p);
return x;
```

Wrong:

```
free(p);
x = p->data; // wrong
```

Why is this wrong?

Because after `free(p)`, the memory no longer belongs to the program. Accessing `p->data` after `free(p)` is unsafe.

Correct rule:

Read the data first. Free the node second.

22. Why Returning `-1` Is Used

The function returns `-1` when deletion fails.

Example:

```
if (index < 1 || index > count(p)) {  
    return -1;  
}
```

This is a simple beginner-friendly way to show an error.

But there is one limitation.

If the linked list can contain negative values, then `-1` might be a valid node value.

Example:

10 -> -1 -> 30 -> NULL

If the function returns `-1`, we may not know whether:

1. deletion failed, or
2. the deleted value was actually `-1`.

For beginner practice, returning `-1` is acceptable.

In more professional code, it is better to return a success or failure flag separately.

23. Common Beginner Mistakes

Mistake 1: Forgetting index validation

Wrong idea:

```
Delete(first, 100);
```

If the function does not validate the index, `p` may become `NULL`.

Then this becomes dangerous:

```
p->next
```

Correct:

```
if (index < 1 || index > count(p)) {  
    return -1;  
}
```

Mistake 2: Using index 0 for deletion

Wrong:

```
Delete(first, 0);
```

In Module 10, deletion starts from index `1`.

Correct idea:

```
Delete index 1 to delete the first node.
```

Mistake 3: Freeing before saving data

Wrong:

```
free(p);  
x = p->data;
```

Correct:

```
x = p->data;  
free(p);
```

Mistake 4: Freeing the first node before moving `first`

Wrong:

```
free(first);  
first = first->next;
```

Correct:

```
q = first;  
first = first->next;  
free(q);
```

Mistake 5: Thinking link change frees memory

This line:

```
q->next = p->next;
```

only removes the node from the chain.

It does not release memory.

You still need:

```
free(p);
```

Without `free(p)`, the node becomes unreachable but still occupies heap memory.

That is called a memory leak.

Mistake 6: Losing the rest of the list

If you delete the first node incorrectly, you may lose access to the rest of the list.

Unsafe idea:

```
first = NULL;
```

This loses the entire list.

Correct head deletion:

```
q = first;  
first = first->next;  
free(q);
```

Mistake 7: Mixing C and C++ memory functions

In C:

```
malloc  
free
```

In C++:

```
new  
delete
```

Do not mix them.

Wrong:

```
Node *p = new Node;  
free(p); // wrong
```

Correct C++:

```
Node *p = new Node;  
delete p;
```

Correct C:

```
struct Node *p = (struct Node *)malloc(sizeof(struct Node));  
free(p);
```

24. Complexity of Deletion

Let n be the number of nodes in the linked list.

Delete first node

Deleting the first node is fast.

Code pattern:

```
q = first;  
first = first->next;  
free(q);
```

This uses only a fixed number of pointer changes.

Time complexity:

$O(1)$

Extra space:

$O(1)$

Delete middle node

To delete a middle node by index, we must move through the list to reach that position.

Time complexity:

$O(n)$

Extra space:

$O(1)$

Delete last node

To delete the last node in a singly linked list, we must reach the node before the last node.

So we need traversal.

Time complexity:

$O(n)$

Extra space:

$O(1)$

Invalid index with validation

Validation uses:

`count(p)`

The `count()` function may visit every node.

So validation takes:

$O(n)$

Complexity table

Case	Time	Extra space	Why
Delete first node	$O(1)$	$O(1)$	Only move <code>first</code> and free one node
Delete middle node	$O(n)$	$O(1)$	Need traversal to reach the position
Delete last node	$O(n)$	$O(1)$	Need traversal to reach the last node
Invalid index with <code>count()</code> validation	$O(n)$	$O(1)$	<code>count()</code> may traverse the list

Important detail:

If we already have a pointer to the previous node, the actual link change is $O(1)$.
But when deleting by index, we usually need $O(n)$ traversal first.

25. Deletion Compared with Insertion

Operation	Main action	Memory action
Insertion	Connect a new node into the list	Allocate memory using <code>malloc</code>
Deletion	Disconnect a node from the list	Free memory using <code>free</code>

Insertion example:

```
t->next = p->next;  
p->next = t;
```

Deletion example:

```
q->next = p->next;  
free(p);
```

Simple comparison:

```
Insertion adds a node to the chain.  
Deletion removes a node from the chain.
```

26. Key Patterns to Memorize

Delete first node

```
q = first;  
x = q->data;  
first = first->next;  
free(q);  
return x;
```

Delete middle or last node

```
q->next = p->next;  
x = p->data;  
free(p);  
return x;
```

Validate index

```
if (index < 1 || index > count(p)) {  
    return -1;  
}
```

Move `p` and `q`

```
for (int i = 0; i < index - 1; i++) {  
    q = p;  
    p = p->next;  
}
```

27. Practice Questions

Question 1

Given this list:

```
5 -> 10 -> 15 -> 20 -> NULL
```

What is the result of:

```
Delete(first, 1);
```

Answer:

```
10 -> 15 -> 20 -> NULL
```

Deleted value:

```
5
```

Question 2

Given this list:

```
5 -> 10 -> 15 -> 20 -> NULL
```

What is the result of:

```
Delete(first, 3);
```

Answer:

```
5 -> 10 -> 20 -> NULL
```

Deleted value:

```
15
```

Question 3

Given this list:

```
5 -> 10 -> 15 -> 20 -> NULL
```

What is the result of:

```
Delete(first, 4);
```

Answer:

```
5 -> 10 -> 15 -> NULL
```

Deleted value:

```
20
```

Question 4

Given this list:

```
5 -> 10 -> 15 -> 20 -> NULL
```

What is the result of:

```
Delete(first, 0);
```

Answer:

```
Invalid index. Return -1.
```

The list remains:

```
5 -> 10 -> 15 -> 20 -> NULL
```

28. Module 10 Final Summary

Module 10 teaches deletion from a singly linked list.

The big idea is:

```
Change links first, then free the removed node.
```

There are two main cases:

1. Delete the first node.
2. Delete any other node.

Deleting the first node is special because `first` must move to the second node.

Middle and last deletion use two pointers:

Pointer	Meaning
<code>p</code>	Node to delete
<code>q</code>	Node before <code>p</code>

The most important line for non-head deletion is:

```
q->next = p->next;
```

This makes the previous node skip the node being deleted.

Then we free the deleted node:

```
free(p);
```

The safest order is:

1. Validate the index.
2. Position the pointers.
3. Save the deleted value.
4. Change the links.
5. Free the deleted node.
6. Return the deleted value.

Final key lesson:

Deleting is not only skipping a node.
A complete deletion must also free the removed node.

You understand Module 10 if you can explain what happens to:

- first
- p
- q
- next
- the deleted node
- the returned value

for head deletion, middle deletion, tail deletion, one-node deletion, empty-list deletion, and invalid indexes.